disco

Density-based Internal Score for Clustering Outcomes

R Translation, Implementation & Experimental Validation

Author: Amin Entezari

amin_entezari@outlook.com

Reference implementation: Beer, Krieger, Weber, Ritzert, Assent, Plant (2025)

DISCO: Internal Evaluation of Density-Based Clustering

arXiv:2503.00127

GitHub: `https://github.com/aminentezari/Disco-R`

R Package: `devtools::install_github("aminentezari/Disco-R")`

March 26, 2026

This document describes the **faithful R translation** of the Python DISCO reference implementation, the three implementation bugs discovered and resolved, and numerical verification across six benchmark datasets.

Contents

1 Introduction **2**

2 Algorithm **2**
2.1 Problem Statement . . . 2
2.2 Step 1: Core Distance . . . 2
2.3 Step 2: Mutual Reachability Distance . . . 2
2.4 Step 3: DC-Distance via Minimum Spanning Tree . . . 2
2.5 Step 4: Scoring Cluster Points . . . 3
2.6 Step 5: Scoring Noise Points . . . 3
2.7 Overall Score . . . 3
2.8 Algorithmic Pipeline Summary . . . 3

3 R Implementation **4**
3.1 Translation Approach . . . 4
3.2 Key Translation Decisions . . . 4
3.3 Installation . . . 4
3.4 Quick Start . . . 4

4 Implementation Bugs Discovered and Fixed **5**
4.1 Bug 1 — Core Distance Off-by-One . . . 5
4.2 Bug 2 — MST Tie-Breaking Difference . . . 6
4.3 Bug 3 — K-Means Cross-Language Non-Determinism . . . 7
4.4 Summary of Bugs . . . 7

5 Experimental Validation **8**
5.1 Datasets . . . 8
5.2 Numerical Results . . . 8
5.3 Interpretation of Differences . . . 9
5.4 Cross-Dataset Comparison: DBSCAN vs K-Means . . . 9

6 API Reference **10**
6.1 `disco_score(X, labels, min_points = 5)` . . . 10
6.2 `disco_samples(X, labels, min_points = 5)` . . . 10
6.3 `p_cluster(X, labels, min_points, precomputed_dc_dists)` . . . 10
6.4 `p_noise(X, labels, min_points, dc_dists)` . . . 10
6.5 `compute_dc_distances(X, min_points = 5)` . . . 10
6.6 Utility Functions . . . 10

7 Score Interpretation Guide **11**

8 Conclusion **11**

1. Introduction

DISCO (Density-based Internal Score for Clustering Outcomes) is a cluster validity index (CVI) introduced by Beer et al. [**?**] that evaluates the quality of density-based clustering results *without requiring ground truth labels.*

Unlike classical CVIs such as the Silhouette Coefficient [**?**] and Davies-Bouldin Index, DISCO:

- Handles **arbitrary-shaped clusters** by replacing Euclidean distance with *density-connectivity distance* (dc-distance);
- Evaluates **noise point quality** explicitly via two sub-scores (p_{sparse} and p_{far});
- Returns a bounded, interpretable score in $[-1, 1]$.

This report documents:

1. The mathematical foundations and algorithmic pipeline (§2);
2. The R implementation and its translation from the Python reference (§3);
3. Three implementation bugs discovered and corrected (§4);
4. Numerical verification across six benchmark datasets (§5);
5. The public API reference (§6).

2. Algorithm

2.1 Problem Statement

Given a dataset $\mathbf{X} = \{x_1, \ldots, x_n\} \subset \mathbb{R}^p$ and a clustering $\mathcal{C} = \{C_1, \ldots, C_K\}$ with optional noise set $\mathcal{N} = \{x_i : \ell_i = -1\}$, DISCO assigns a quality score $\rho(x_i) \in [-1, 1]$ to every point and returns the mean as the overall score.

2.2 Step 1: Core Distance

The *core distance* of a point x_i with parameter μ (min_points) is the distance to its $(\mu - 1)$-th nearest neighbour:

$$\kappa_\mu(x_i) = d\big(x_i,\, x_i^{(\mu-1)}\big)$$

A small core distance indicates a dense region; a large core distance indicates a sparse region.

2.3 Step 2: Mutual Reachability Distance

$$d_{\text{reach}}(x_i, x_j) = \max\big(\kappa_\mu(x_i),\ \kappa_\mu(x_j),\ d(x_i, x_j)\big)$$

This smooths distances near cluster cores and preserves large gaps between clusters.

2.4 Step 3: DC-Distance via Minimum Spanning Tree

1. Construct the complete graph G with edge weights d_{reach}.
2. Compute the Minimum Spanning Tree T of G using Prim's algorithm.
3. The **dc-distance** between x_i and x_j is the maximum edge weight on the unique path between them in T:

$$d_{\text{DC}}(x_i, x_j) = \max_{e \in \text{path}_T(i,j)} w(e)$$

This is the *minimax* (bottleneck) path distance and captures density-connectivity: two points in the same dense cluster have a small dc-distance, while points in different clusters must cross a sparse gap.

2.5 Step 4: Scoring Cluster Points

For each non-noise point $x_i \in C_{c_i}$, DISCO uses a silhouette-style formula on dc-distances:

$$a(i) = \frac{1}{|C_{c_i}| - 1} \sum_{j \in C_{c_i}, j \neq i} d_{\mathrm{DC}}(x_i, x_j) \tag{1}$$

$$b(i) = \min_{c \neq c_i} \frac{1}{|C_c|} \sum_{j \in C_c} d_{\mathrm{DC}}(x_i, x_j) \tag{2}$$

$$\rho_{\mathrm{cluster}}(x_i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \tag{3}$$

2.6 Step 5: Scoring Noise Points

For each noise point $x_n \in \mathcal{N}$, let $M_c = \max_{j \in C_c} \kappa_\mu(x_j)$ be the maximum core distance in cluster c. Two sub-scores are computed and the minimum taken:

Sparsity score

$$p_{\mathrm{sparse}}(x_n) = \min_c \frac{\kappa_\mu(x_n) - M_c}{\max(\kappa_\mu(x_n),\ M_c)}$$

Remoteness score

$$p_{\mathrm{far}}(x_n) = \min_c \frac{d_{\min,c}(x_n) - M_c}{\max(d_{\min,c}(x_n),\ M_c)}$$

where $d_{\min,c}(x_n) = \min_{j \in C_c} d_{\mathrm{DC}}(x_n, x_j)$.

$$\rho_{\mathrm{noise}}(x_n) = \min\big(p_{\mathrm{sparse}}(x_n),\ p_{\mathrm{far}}(x_n)\big)$$

2.7 Overall Score

$$\mathrm{DISCO}(\mathbf{X}, \mathcal{C}) = \frac{1}{n} \sum_{i=1}^{n} \rho(x_i), \qquad \rho(x_i) = \begin{cases} \rho_{\mathrm{cluster}}(x_i) & x_i \notin \mathcal{N} \\ \rho_{\mathrm{noise}}(x_i) & x_i \in \mathcal{N} \end{cases}$$

2.8 Algorithmic Pipeline Summary

Step	Function (R)	Operation
1	`calculate_reachability_distance`	Core distances + mutual reachability matrix
2	`get_mst_edges`	Prim's MST on mutual reachability graph
3	`extract_dc_distances_from_mst`	BFS minimax path extraction
4	`compute_dc_distances`	Full $n \times n$ DC-distance matrix
5	`p_cluster`	Silhouette-style scores for cluster points
6	`p_noise`	Sparsity + remoteness scores for noise
7	`disco_samples`	Per-point DISCO scores
8	`disco_score`	Mean over all points

3. R Implementation

3.1 Translation Approach

The R package `disco` is a **line-by-line translation** of the Python reference implementation (`disco.py` + `dctree.py`). Every function in R mirrors the corresponding Python function, with identical logic, identical edge-case handling, and identical numerical results (to floating-point precision).

The package consists of a single file `R/disco.R` combining three logical sections:

Section	Source	Functions
DC-Distance	`dctree.py`	`calculate_reachability_distance`, `get_mst_edges`, `extract_dc_distances_from_mst`, `compute_dc_distances`
DISCO Metric	`disco.py`	`disco_score`, `disco_samples`, `p_cluster`, `p_noise`
Utilities	—	`make_circles`, `make_moons`, `make_blobs`, `plot_disco_scores`, `compare_clusterings`, `summary_disco_scores`

3.2 Key Translation Decisions

k-NN library

Python uses `sklearn.neighbors.KDTree`; R uses `FNN::get.knn`. Both exclude the query point itself from results.

MST construction

Python uses a custom Prim's implementation (`dctree.py` lines 401–438). R replicates this exactly rather than using `igraph::mst` (see Bug 2 in §4).

Silhouette computation

Python delegates to `sklearn.metrics.silhouette_samples` with `metric="precomputed"`. R uses an explicit loop implementing the same formula (Equations 1–3).

3.3 Installation

```r
# Install from GitHub
devtools::install_github("aminentezari/Disco-R")

# Load
library(disco)
```

3.4 Quick Start

```r
library(disco)
library(dbscan)

# Generate benchmark data
data   <- make_circles(n_samples = 300, noise = 0.05,
                       factor = 0.4, random_state = 42)
X      <- data$X
```

```r
# Cluster
db_labels <- dbscan(X, eps = 0.2, minPts = 5)$cluster - 1L
km_labels <- kmeans(X, centers = 2, nstart = 10)$cluster - 1L

# Evaluate
disco_score(X, db_labels)   # 0.6900921813  (high: correct)
disco_score(X, km_labels)   # 0.0270091908  (low: wrong shape)

# Per-point scores
scores <- disco_samples(X, db_labels)
summary_disco_scores(scores)

# Compare algorithms
compare_clusterings(X, list(DBSCAN = db_labels, KMeans = km_labels)
    )
```

4. Implementation Bugs Discovered and Fixed

Three numerical differences were found between the initial R translation and the Python reference. Each is documented with its root cause and exact fix.

4.1 Bug 1 — Core Distance Off-by-One

Location

`calculate_reachability_distance()` and `p_noise()`

Root Cause

Python (`dctree.py`, line 710) computes core distances as:

```python
reach_dists[i] = np.max(np.partition(eucl_dists[i], min_points)[:
    min_points])
```

`eucl_dists[i]` is a row of the *full* pairwise distance matrix, which **includes the point itself at distance 0**. Therefore `np.partition(..., 5)[:5]` returns $[0, d_1, d_2, d_3, d_4]$ and the maximum is d_4 — the **4th nearest neighbour**.

R's `FNN::get.knn(X, k = min_points)` **excludes the point itself**, so `k = 5` returns the 5th nearest neighbour — one neighbour further than Python.

Effect

Core distances were systematically larger in R, inflating mutual reachability distances and changing the MST and all dc-distances.

Fix

```r
# BEFORE (wrong)
knn_result <- FNN::get.knn(X, k = min_points)
core_dists <- knn_result$nn.dist[, min_points]

# AFTER (correct)
```

```
k           <- min_points - 1    # matches Python's self-inclusive
    partition
knn_result <- FNN::get.knn(X, k = k)
core_dists <- knn_result$nn.dist[, k]
```

4.2 Bug 2 — MST Tie-Breaking Difference

Location

`compute_dc_distances()` — MST construction

Root Cause

When two candidate edges in Prim's algorithm have **equal weight** (which is frequent because many distances get floored to the core distance value), the choice of which edge to add next depends on the tie-breaking rule:

- **Python** uses a custom Prim's implementation (`dctree.py`, lines 401–438) with `np.argmin(nodes_min_di`
- **R (initial)** used `igraph::mst(algorithm = "prim")` which has different internal ordering.

Different tie-breaking ⇒ different MST ⇒ different dc-distances ⇒ different DISCO scores.

Effect

After fixing Bug 1, DBSCAN scores matched (0.6900921813) but K-Means still differed (R: 0.0257 vs Python: 0.0270). K-Means is more sensitive because its clusters span both rings, creating more equal-weight boundary edges.

Fix

Replaced `igraph::mst` with an exact R port of Python's `_get_mst_edges()`, line-by-line:

```
get_mst_edges <- function(dist_matrix) {
  n               <- nrow(dist_matrix)
  nodes_min_dist <- rep(Inf, n)
  parent          <- rep(1L, n)
  not_in_mst      <- rep(TRUE, n)
  u               <- 1L            # Python starts at node 0; R at 1
  nodes_min_dist[u] <- 0
  not_in_mst[u]     <- FALSE

  for (step in seq_len(n - 1)) {
    update_mask <- not_in_mst & (dist_matrix[u, ] < nodes_min_dist)
    if (any(update_mask)) {
      nodes_min_dist[update_mask] <- dist_matrix[u, update_mask]
      parent[update_mask]         <- u
    }
    candidates <- which(not_in_mst)
    u          <- candidates[which.min(nodes_min_dist[candidates])]
    mst_i[step]    <- parent[u]
    mst_j[step]    <- u
    mst_dist[step] <- nodes_min_dist[u]
    not_in_mst[u]  <- FALSE
  }
  data.frame(i = mst_i, j = mst_j, dist = mst_dist)
}
```

4.3 Bug 3 — K-Means Cross-Language Non-Determinism

Location

All experiment scripts — K-Means clustering section

Root Cause

`set.seed(42)` in R and `random_state=42` in Python control **different random number generators**. Both use Mersenne Twister but with different implementations, producing different sequences from the same seed. K-Means centroid initialisation therefore differed:

	Cluster 0	Cluster 1	Centroid 0	Centroid 1
Python	144 pts	156 pts	(0.462, 0.114)	(−0.402, −0.172)
R	148 pts	152 pts	(0.460, 0.077)	(−0.423, −0.143)

The two runs converged to different local optima.

Effect

DISCO evaluated on different partitions — scores can never match regardless of implementation correctness.

Fix

Export Python's final labels as CSV and load in R:

```
# Python: export labels
pd.DataFrame({"km_label": km.labels_}).to_csv("python_km_labels.csv
    ",
                                                index=False)
```

```
# R: load Python labels
km_labels <- as.integer(
  read.csv("python_km_labels.csv")$km_label
)
```

This guarantees both languages evaluate DISCO on the **identical partition**.

4.4 Summary of Bugs

#	Location	Root Cause	Fix
1	`calculate_reachability_distance`, `p_noise`	Python includes self in k-NN; FNN excludes self	`k = min_points - 1`
2	`get_mst_edges`	`igraph::mst` tie-breaking differs from Python's Prim	Port Python's exact `_get_mst_edges()`
3	All experiment scripts	R and Python RNGs produce different K-Means partitions	Export Python labels as CSV; load in R

5. Experimental Validation

Six benchmark datasets were used to validate the R implementation against the Python reference. For each dataset, both DBSCAN and K-Means clusterings were evaluated.

5.1 Datasets

Dataset	Source	n	Clusters	Shape	DBSCAN eps
Concentric Circles	Generated	300	2	Rings	0.20
Two Moons	Generated	300	2	Crescents	0.20
Complex9	`complex9.arff`	3031	9	Mixed	15.0
Complex8	`complex8.arff`	2551	8	Mixed	10.0
Dartboard1	`dartboard1.arff`	1000	4	Rings	0.08
3-Spiral	`3-spiral.arff`	312	3	Spirals	2.0

5.2 Numerical Results

All results are shown to 20 decimal places. Differences are within machine epsilon ($\approx 2.2 \times 10^{-16}$) for all datasets.

Concentric Circles

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	`0.69009218128208216214`	`0.69009218128208293930`	7.77×10^{-16}
K-Means	`0.02700919079440020423`	`0.02700919079439973586`	4.68×10^{-16}

Two Moons

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	`0.70122811114499916663`	`0.70122811114499850049`	6.66×10^{-16}
K-Means	`0.22303566235762423142`	`0.22303566235762423142`	**0**

Complex9

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	`0.42094645142453540609`	`0.42094645142454312214`	7.72×10^{-16}
K-Means	`0.05510967752575094553`	`0.05510967752573882328`	1.21×10^{-15}

Complex8

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	`0.07186876516496862322`	`0.07186876516500724510`	3.86×10^{-15}
K-Means	`0.01225747486413241577`	`0.01225747486413229781`	1.18×10^{-16}

Dartboard1

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	`0.8743426860803062972 1`	`0.8743426860803125144 6`	6.22×10^{-15}
K-Means	`−0.0033636710500706589 3`	`−0.0033636710500714972 3`	8.38×10^{-16}

3-Spiral

Method	R	Python	$\lvert\Delta\rvert$
DBSCAN	`0.5836514046464754690 3`	`0.5836514046464750249 4`	4.44×10^{-16}
K-Means	`−0.0014176358526362101 1`	`−0.0014176358526362020 8`	8.03×10^{-18}

5.3 Interpretation of Differences

Magnitude	Interpretation
$= 0$	Exact bit-for-bit agreement
$< 10^{-14}$	Normal IEEE 754 floating-point rounding; implementations are **numerically equivalent**
$\geq 10^{-12}$	Potential algorithmic discrepancy worth investigating

The largest observed difference is 6.22×10^{-15} (Dartboard1 DBSCAN), well within the range of normal floating-point accumulation across $n = 1000$ pairwise distance computations. All differences are **at least two orders of magnitude below the DISCO score itself**, confirming the R implementation is faithful.

5.4 Cross-Dataset Comparison: DBSCAN vs K-Means

Dataset	DISCO (DBSCAN)	DISCO (K-Means)	Observation
Concentric Circles	0.6901	0.0270	K-Means cuts through rings; DBSCAN respects them
Two Moons	0.7012	0.2230	K-Means partially captures moons via centroid split
Complex9	0.4209	0.0551	Mixed shapes favour density-based approach
Complex8	0.0719	0.0123	Low overall due to density variation
Dartboard1	0.8743	−0.0034	4 concentric rings: K-Means actively harmful
3-Spiral	0.5837	−0.0014	Spiral shape impossible for K-Means

Remark 1. *Negative K-Means scores on Dartboard1 and 3-Spiral are* ***correct and expected****. They indicate that K-Means assigns points to clusters that are further away in dc-distance space than the neighbouring cluster — a meaningful penalty that traditional CVIs (Silhouette, Davies-Bouldin) would not assign.*

6. API Reference

6.1 `disco_score(X, labels, min_points = 5)`

Returns a single scalar summarising overall clustering quality.

Argument	Type	Description
`X`	matrix	$n \times p$ numeric feature matrix
`labels`	integer vec.	Cluster labels; `-1` for noise
`min_points`	integer	Neighbourhood size ≥ 2. Default: `5`

Returns: Single `numeric` in $[-1, 1]$.

```
disco_score(X, db_labels)    # 0.6900921813
```

6.2 `disco_samples(X, labels, min_points = 5)`

Returns per-point DISCO scores.

Returns: `numeric` vector of length n.

```
scores <- disco_samples(X, db_labels)
which(scores < 0)    # misassigned points
```

6.3 `p_cluster(X, labels, min_points, precomputed_dc_dists)`

Silhouette-style scores using dc-distances. Note: `-1` is treated as a valid cluster label here, not noise.

6.4 `p_noise(X, labels, min_points, dc_dists)`

Returns `list(p_sparse, p_far)` for noise points only.

6.5 `compute_dc_distances(X, min_points = 5)`

Returns the $n \times n$ DC-distance matrix. Useful for precomputing once and reusing across multiple clusterings.

```
D <- compute_dc_distances(X, min_points = 5)
s1 <- p_cluster(D, labels1, precomputed_dc_dists = TRUE)
s2 <- p_cluster(D, labels2, precomputed_dc_dists = TRUE)
```

6.6 Utility Functions

Function	Description
`make_circles(n, noise, factor, seed)`	Concentric circles
`make_moons(n, noise, seed)`	Two interleaving crescents
`make_blobs(n, centers, std, seed)`	Gaussian blobs
`plot_disco_scores(X, labels, scores)`	Red–yellow–green scatter plot
`compare_clusterings(X, list(...))`	Rank multiple clusterings by DISCO
`summary_disco_scores(scores)`	Mean, median, SD, quantiles, etc.

7. Score Interpretation Guide

Score Range	Label	Meaning
$[0.7,\ 1.0]$	Excellent	Well-separated, compact clusters
$[0.4,\ 0.7)$	Good	Clear cluster structure
$[0.0,\ 0.4)$	Moderate	Overlapping or touching clusters
$[-0.3,\ 0.0)$	Poor	Misassigned points
$[-1.0,\ -0.3)$	Very poor	Clustering worse than random

Common pitfall. `dbscan()` in R uses `0` for noise, not `-1`:

```
# WRONG: dbscan returns 0 for noise
labels <- dbscan(X, eps = 0.2)$cluster

# CORRECT: convert to -1
labels <- dbscan(X, eps = 0.2)$cluster - 1L
```

8. Conclusion

The `disco` R package is a faithful translation of the Python DISCO reference implementation. Three implementation bugs were identified and corrected, each with a clearly documented root cause and fix. Numerical validation across six benchmark datasets confirms that the R and Python implementations produce identical results to within normal IEEE 754 floating-point rounding ($< 10^{-14}$).

The experimental results demonstrate that DISCO correctly discriminates between well-suited and ill-suited clustering algorithms: density-based methods (DBSCAN) consistently outperform centroid-based methods (K-Means) on non-convex datasets, and negative scores meaningfully penalise algorithms that cut through genuine cluster structure.

References

[1] Beer, A., Krieger, L., Weber, P., Ritzert, M., Assent, I., Plant, C. (2025). *DISCO: Internal Evaluation of Density-Based Clustering.* arXiv:2503.00127 [cs.LG].

[2] Rousseeuw, P.J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.

[3] Ester, M., Kriegel, H.P., Sander, J., Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of KDD*, 96(34), 226–231.

[4] Davies, D.L., Bouldin, D.W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1(2), 224–227.

[5] R Core Team (2023). *R: A Language and Environment for Statistical Computing.* R Foundation for Statistical Computing, Vienna, Austria.

[6] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.